

M_01_Excercise

Name: Praveen Kumar

Emp ID: 90119740

Database: PostgreSQL - icicibank

Q1. ICICI Bank's Baner Pune branch stores customer addresses in a spreadsheet, and the credit card department keeps its own copy of the same addresses. A customer moves house and tells only the branch. Name the two file-system problems this creates, and explain in two sentences how a database prevents both.

The two problems are **data redundancy** and **data inconsistency**. The same customer information is stored in more than one place, so different copies can have different values. A database stores shared information in a controlled central system, so an approved change can be updated once and used by all departments.

Q2. State three things a DBMS does that a file system does not. For each, give one concrete banking consequence of not having it.

- 1. Concurrency control:** It manages simultaneous updates; otherwise two ATMs could process withdrawals using the same old balance.
- 2. Transaction management:** It makes related operations succeed or fail together; otherwise money could be debited without being credited.
- 3. Data integrity:** It enforces rules such as keys and constraints; otherwise invalid account-customer links or invalid balances could be stored.

Q3. Fill in the table with the matching term.

Relation: Table - example: customer.

Tuple: Row/record - one customer record.

Attribute: Column/field - for example customer_id.

Degree: Number of columns - loan has 11 columns.

Cardinality: Number of rows - customer has 30 rows in the supplied data.

Q4. For each of the following, say whether it is a primary key, a candidate key, a foreign key, or none of these.

- 1. customer.customer_id** - Primary key
- 2. account.account_number** - Candidate key
- 3. account.customer_id** - Foreign key
- 4. account.balance** - None
- 5. employee.manager_id** - Foreign key
- 6. branch.ifsc_code** - Candidate key

Q5. Classify each workload as OLTP or OLAP, and give a one-line reason.

- 1. Mobile app balance check** - **OLTP:** operational, frequent access.
- 2. NPL by branch for the quarter** - **OLAP:** historical analysis and aggregation.
- 3. ATM dispenses Rs 10,000** - **OLTP:** real-time transaction processing.
- 4. Regulator asks total value of transactions above Rs 200,000 for last FY** - **OLAP:** large historical

aggregation.

5. Teller opens a savings account - OLTP: operational insert/update.

6. Marketing finds top 100 by average monthly balance - OLAP: analytical calculation over many records.

Q6. ICICI Bank's head of reporting asks to run the quarterly regulatory report directly against the core banking Oracle database. Give three technical reasons to refuse, and name what you would offer instead.

The report can use significant CPU, memory and I/O on the core system. Long-running analytical queries can also compete with banking transactions for resources and increase transaction latency. It can affect the availability and performance of the core banking workload. I would offer a separate reporting or data-warehouse environment.

Q7. Match each ACID property to the failure it prevents.

Atomicity -> **d**: Money is debited but never credited.

Consistency -> **c**: An account exists for a customer who does not.

Isolation -> **b**: Two ATMs both pay out against the same balance.

Durability -> **a**: A committed transfer disappears after a power cut.

Q8. Explain in your own words why the write-ahead log is written to disk before the data page. What would break if the order were reversed?

The WAL is written first so the information needed for crash recovery is safely stored before the actual data page is written. If the data page were written first and the system crashed before the WAL record was durable, PostgreSQL might not have enough information to recover the change correctly.

Q9. A junior developer writes `SELECT * FROM customer` in the internet banking application. Explain which type of data independence this breaks and what will go wrong when the bank later adds a `pan_number` column.

It breaks **logical data independence** because the application depends on the exact set of columns in the table. When `pan_number` is added, `SELECT *` returns an additional column and can change the result structure expected by the application.

Q10. In one sentence each, state where you would use Oracle, PostgreSQL, MySQL and Redis at ICICI Bank, and why.

Oracle: Enterprise/core banking workloads where Oracle is the established relational database.

PostgreSQL: Open-source relational and data-engineering workloads.

MySQL: Application-facing relational workloads where MySQL is suitable.

Redis: Fast in-memory access such as caching, sessions and frequently accessed temporary data.

Q11. Write a query that returns the database name, the current user and the server port.

```
SELECT current_database(),
       current_user,
       inet_server_port();
```

Q12. Using `information_schema`, list every column of the loan table with its data type and whether it accepts NULL. How many columns does loan have? What is the degree of the relation?

```

SELECT column_name,
       data_type,
       is_nullable
FROM information_schema.columns
WHERE table_schema = 'public'
      AND table_name = 'loan'
ORDER BY ordinal_position;

```

Q13. Report the number of rows in all nine tables in a single result set, sorted by table name.

```

SELECT 'account' AS table_name, COUNT(*) AS row_count FROM account
UNION ALL SELECT 'account_txn', COUNT(*) FROM account_txn
UNION ALL SELECT 'beneficiary', COUNT(*) FROM beneficiary
UNION ALL SELECT 'branch', COUNT(*) FROM branch
UNION ALL SELECT 'card', COUNT(*) FROM card
UNION ALL SELECT 'customer', COUNT(*) FROM customer
UNION ALL SELECT 'employee', COUNT(*) FROM employee
UNION ALL SELECT 'loan', COUNT(*) FROM loan
UNION ALL SELECT 'loan_payment', COUNT(*) FROM loan_payment
ORDER BY table_name;

```

Q14. List every foreign key in the database, showing the child table, the child column, and the parent table it points to. Which table has a foreign key pointing at itself, and what does that represent in the bank?

```

SELECT tc.table_name AS child_table,
       kcu.column_name AS child_column,
       ccu.table_name AS parent_table,
       ccu.column_name AS parent_column
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu
  ON tc.constraint_name = kcu.constraint_name
 AND tc.table_schema = kcu.table_schema
JOIN information_schema.constraint_column_usage AS ccu
  ON tc.constraint_name = ccu.constraint_name
 AND tc.table_schema = ccu.table_schema
WHERE tc.constraint_type = 'FOREIGN KEY'
      AND tc.table_schema = 'public'
ORDER BY child_table, child_column;

```

Q15. Find the three largest tables on disk, including indexes, with sizes in a human-readable form. Why is account_txn the largest?

```

SELECT relname AS table_name,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 3;

```

Q16. ICICI Bank opens a current account for a new corporate customer. Insert the customer and the account using the given details, then read both rows back.

```

BEGIN;

INSERT INTO customer
(customer_id, first_name, last_name, date_of_birth, email, phone, city,
 kyc_status, risk_rating, customer_since, home_branch_id)
VALUES
(1032, 'Latha', 'Krishnan', DATE '1985-09-23',
 'latha.krishnan@example.com', '9845099887', 'Chennai',
 'VERIFIED', 'MEDIUM', DATE '2026-03-01', 5);

INSERT INTO account
(account_id, account_number, customer_id, branch_id, account_type,
 balance, currency, status, opened_date, interest_rate)
VALUES

```

```
(9047, 'ICIC0000009047', 1032, 5, 'CURRENT',
500000.00, 'INR', 'ACTIVE', DATE '2026-03-01', 0.00);
```

```
SELECT * FROM customer
WHERE customer_id = 1032;
```

```
SELECT * FROM account
WHERE account_id = 9047;
```

Q17. The customer deposits Rs 75,000 and then withdraws Rs 20,000. Write both updates so that they compute from the current balance rather than assigning an absolute figure, then confirm the final balance is 555,000.

```
UPDATE account
SET balance = balance + 75000
WHERE account_id = 9047;
```

```
UPDATE account
SET balance = balance - 20000
WHERE account_id = 9047;
```

```
SELECT account_id, account_number, balance
FROM account
WHERE account_id = 9047;
```

```
ROLLBACK;
```

Q18. Try to delete customer 1032 without deleting the account first. Record the error. Then delete both rows in the correct order and confirm the counts.

```
BEGIN;
```

```
INSERT INTO customer
(customer_id, first_name, last_name, date_of_birth, email, phone, city,
kyc_status, risk_rating, customer_since, home_branch_id)
VALUES
(1032, 'Latha', 'Krishnan', DATE '1985-09-23',
'latha.krishnan@example.com', '9845099887', 'Chennai',
'VERIFIED', 'MEDIUM', DATE '2026-03-01', 5);
```

```
INSERT INTO account
(account_id, account_number, customer_id, branch_id, account_type,
balance, currency, status, opened_date, interest_rate)
VALUES
(9047, 'ICIC0000009047', 1032, 5, 'CURRENT',
500000.00, 'INR', 'ACTIVE', DATE '2026-03-01', 0.00);
```

```
SAVEPOINT before_delete;
```

```
DELETE FROM customer
WHERE customer_id = 1032;
```

```
ROLLBACK TO SAVEPOINT before_delete;
```

```
DELETE FROM account
WHERE account_id = 9047;
```

```
DELETE FROM customer
WHERE customer_id = 1032;
```

```
SELECT COUNT(*) AS customer_count FROM customer;
SELECT COUNT(*) AS account_count FROM account;
```

```
ROLLBACK;
```

Q19. Write three statements that the database must reject, one for each of these reasons: FIXED account type, negative transaction amount, and IN_PROGRESS

KYC status.

```
-- 1. Invalid account_type
BEGIN;
INSERT INTO account
(account_id, account_number, customer_id, branch_id, account_type,
 balance, currency, status, opened_date, interest_rate)
VALUES
(9999, 'ICIC0000009999', 1001, 1, 'FIXED',
 1000.00, 'INR', 'ACTIVE', CURRENT_DATE, 0.00);
ROLLBACK;

-- 2. Negative transaction amount
BEGIN;
INSERT INTO account_txn
(txn_id, account_id, txn_date, txn_type, amount, channel,
 narration, status, reference_no)
VALUES
(999999, 9001, CURRENT_TIMESTAMP, 'DEBIT', -5000.00,
 'ATM', 'Test transaction', 'SUCCESS', 'TEST999999');
ROLLBACK;

-- 3. Invalid KYC status
BEGIN;
INSERT INTO customer
(customer_id, first_name, last_name, date_of_birth, email, phone, city,
 kyc_status, risk_rating, customer_since, home_branch_id)
VALUES
(9999, 'Test', 'Customer', DATE '1990-01-01',
 'test9999@example.com', '9999999999', 'Mumbai',
 'IN_PROGRESS', 'MEDIUM', CURRENT_DATE, 1);
ROLLBACK;
```

Q20. Using two sessions, demonstrate that an uncommitted UPDATE in Session A is invisible to Session B.

```
-- SESSION A
BEGIN;

UPDATE account
SET balance = balance + 1000
WHERE account_id = 9004;

SELECT account_id, balance
FROM account
WHERE account_id = 9004;

-- SESSION B
SELECT account_id, balance
FROM account
WHERE account_id = 9004;

-- SESSION A
ROLLBACK;

-- SESSION B
SELECT account_id, balance
FROM account
WHERE account_id = 9004;
```

Q21. In a single transaction, transfer Rs 12,000 from account 9004 to account 9010, then roll it back. Prove with a query before and after that neither balance changed.

```
SELECT account_id, balance
FROM account
```

```

WHERE account_id IN (9004, 9010)
ORDER BY account_id;

BEGIN;

UPDATE account
SET balance = balance - 12000
WHERE account_id = 9004;

UPDATE account
SET balance = balance + 12000
WHERE account_id = 9010;

SELECT account_id, balance
FROM account
WHERE account_id IN (9004, 9010)
ORDER BY account_id;

ROLLBACK;

SELECT account_id, balance
FROM account
WHERE account_id IN (9004, 9010)
ORDER BY account_id;

```

Q22. Show the current values of shared_buffers, work_mem, wal_level and fsync. For each, write one sentence explaining what it controls.

```

SELECT name, setting
FROM pg_settings
WHERE name IN ('shared_buffers', 'work_mem', 'wal_level', 'fsync')
ORDER BY name;

```

Q23. UPI payments may peak at 4,000 transactions per second. The vendor proposes MySQL MyISAM because it is faster for reads. Write the response.

MyISAM is not suitable for a payment transaction store because it does not provide the transaction and crash-recovery guarantees expected for this type of workload. The specific alternative is **InnoDB**, which supports transactions, row-level locking and crash recovery.

Q24. The bank has 40 million rows in account_txn and growing by 2 million a month. The branch operations team wants a screen showing every transaction for an account, and the risk team wants monthly totals across all accounts. Explain why one database design cannot serve both well, and describe the two-system arrangement.

The account screen needs fast selective queries for day-to-day operations, while monthly totals require large scans and aggregations across historical data. Running both workloads on the same system can make the operational screen slower. I would keep the operational database for transactions and use a separate data warehouse/reporting system for analytics. In this programme, Module 10 builds **ICICIBankDW** for the warehouse side.

Q25. During an incident review, a developer says: "The application logged 'transfer successful', so the money must have moved." Explain why that statement is not evidence, and state what would be.

An application log only shows that the application wrote that message; it does not prove that the database transaction committed. Evidence would be the committed database transaction, the resulting account balances and the corresponding transaction records/reference.

Q26. The DBA proposes setting fsync = off on the core banking PostgreSQL server because it makes overnight batch loads faster. Write a short recommendation.

I would not set **fsync = off** on the core banking server because it increases the risk of data loss or corruption after a system failure. It may be used only in a disposable development or test environment where the data can be recreated and the risk is understood.

Q27. A new joiner argues that check constraints such as balance >= 0 belong in the application, not the database. Give two arguments for keeping the rule in the database, and one situation where the application layer is the right place.

First, a database constraint protects the rule regardless of which application or job changes the data. Second, it keeps the basic data-integrity rule in one central place and prevents invalid balances from being stored. The application layer is better for complex workflow rules that depend on external services, user interaction or broader business decisions.

Q28. ICICI Bank is acquiring a smaller bank whose core system runs on PostgreSQL, while ICICI runs Oracle. List four differences between the two products that you would expect to cause problems during the merge.

1. **Data types:** PostgreSQL and Oracle have different type names and behaviour in some areas.
2. **SQL syntax and functions:** date functions and other built-in functions can differ.
3. **Procedural languages:** PostgreSQL PL/pgSQL and Oracle PL/SQL are different.
4. **Database-specific features:** indexing, sequences/identity behaviour and other platform-specific features may need changes.

Q29. Reply to ticket ICICI-4417 in language a non-technical banker will understand. Explain what moving from spreadsheets to a relational database will fix, what it will not fix, and what the bank must do differently as a result.

Moving from spreadsheets to a database gives the bank one controlled place for shared customer and account information. It reduces duplicate copies of the same information across departments. When an approved change is made, users can work from the same updated record. The database can prevent invalid links, such as an account referring to a customer who does not exist. It can also prevent values such as a negative account balance when that rule is defined. A multi-step banking transaction can be handled as one complete operation instead of leaving only part of the change. However, the database will not automatically correct wrong information entered by staff. It will not by itself solve poor processes, weak access control or poor data governance. The bank therefore needs clear data ownership, controlled access and proper procedures for entering and checking information. Large reports should be separated from the day-to-day transaction workload. Backups, monitoring and recovery testing are also required.